arXiv:2604.12102v3 [cs.AI] 30 Sep 2026

Spatial Atlas: Compute-Grounded Reasoning for Spatial-Aware Research Agent Benchmarks

Arun Sharma
University of Minnesota
arunshar@umn.edu

Abstract

We describe compute-grounded reasoning (CGR), a design pattern in which code computes selected sub-problems from explicit intermediate representations before a language model answers. Spatial Atlas implements CGR as an Agent2Agent (A2A) server with a spatial question-answering handler and a machine-learning engineering handler. The spatial handler asks a language model to extract a scene graph, and code then fills in missing distances and checks the extracted safety rules. A separate benchmark driver can also run a strict metric bridge. It computes the gap for horizontal-gap questions from segmentation masks and a reconstructed point map, and it passes that gap to the answering model as a fact. The bridge returns a fixed unavailable answer when an evidence check fails, and it never falls back to model-estimated coordinates. The ML-engineering handler generates pipeline code and parses validation scores, and it caps the number of repair and refinement passes. Its code execution is off by default. The repository also provides four run modes that can write label-free journals, a shuffled-image control mapping, and journal validators that reject label-bearing fields. We report one private label-free operational run in which four paths each wrote eight prediction rows with zero retries. Labels stayed sealed and no score was computed, so this run establishes operational integrity only. We report no FieldWorkArena result, because the benchmark data were not accessible. We also omit every performance, latency, and resource-use number that lacks a reproducible run artifact.

1 Introduction

GPT-4 performs at a human level on various professional and academic benchmarks [25]. Building reliable autonomous agents on large language models (LLMs) remains an open problem [31]. Two recent benchmarks cover different parts of this problem. FieldWorkArena [30] evaluates agents on field-work tasks built from images and videos captured in factories, warehouses, and retail sites. MLE-bench [5] tests end-to-end machine learning engineering on 75 Kaggle competitions.

This paper describes one server that hosts a spatial question-answering handler and an ML-engineering handler. The two handlers share a model client, model tiers, and usage tracking. Both kinds of task contain sub-steps that code can compute, such as a distance between two objects or a validation score.

We present **Spatial Atlas**, a research agent that exposes both handlers through a single Agent2Agent (A2A) protocol server [13]. FieldWorkArena motivated the original spatial adapter, but the benchmark is not part of the reported evaluation because its data were not accessible. The system follows a design pattern that we call *compute-grounded reasoning* (CGR). When a sub-problem has a deterministic solution, code computes it first and passes the result to the language model as a fact. The architecture has the seven components below.

1. **Spatial Scene Graph Engine.** The Strong tier extracts entities and relations from the file descriptions. Code computes the distances the model did not supply, checks the extracted safety

rules, and places the results in the prompt. The graph does not record whether a stored distance came from the model or from code, and perception errors can still propagate into it.
2. **Confidence-Gated Refinement.** One Fast-tier self-grade decides whether the Strong tier writes a refinement, and the controller never refines more than once. The codebase also defines an expected-information-gain action selector, but no code in the repository calls it. The effect of this refinement on accuracy and resource use has not been measured.
3. **Fail-Closed ML Pipeline.** This component generates strategy-aware code, and it runs that code only after an execution opt-in, an isolated-worker attestation, and authenticated server startup. The initial script and its repairs get at most 3 attempts, and score-driven refinement adds at most 2 more runs. Every run has a 600-second timeout, and dummy submissions require a separate opt-in.
4. **Score-Driven Refinement.** An iterative loop parses machine-readable validation scores, asks the Strong tier for a revision, and keeps the revision only when its parsed score is better.
5. **Leak Audit and Hint Registry.** A fixed audit prompt asks generated pipelines to check four common train and test leakage patterns, and a registry adds a targeted hint for its one registered competition.
6. **Strict Metric Perception Bridge.** For horizontal-gap questions, the bridge computes the gap between two objects from segmentation masks and a reconstructed point map, and it passes that gap to the model as a fact. It records provenance and evidence digests, and it returns an explicit unavailable answer when an evidence check fails. Only the frozen benchmark driver reaches it.
7. **Label-Free Run Modes and Validators.** The driver provides four run modes, a digest-bound shuffled-image control mapping with no fixed point, and append-only journals that reject label-bearing fields.

The scene-graph engine and the strict metric bridge carry out compute-grounded reasoning most directly. The scene-graph engine computes distances and rule violations, and the strict metric bridge computes a reconstructed surface gap. The system writes these values into the prompt before the model answers, so an operator who records the prompt can inspect them. Their effect on reliability, accuracy, latency, and resource use must still be established through the planned evaluation.

Sections 3 to 7 describe the system as the released code implements it. Section 8 states the evidence boundary and reports one label-free operational run, and Section 9 gives the evaluation that remains to be done. Section 10 lists the limitations.

2 Related Work

Agent frameworks. AutoGPT [29] was an early open-source agent that chains LLM calls toward a user-set goal. OpenHands [32] is an open platform for software-development agents, and it was first released under the name OpenDevin. SWE-bench [18] tests whether language models can resolve real GitHub issues, and its authors report that the best model they evaluated resolved only a small share of them. SWE-agent [35] adds an agent-computer interface for the same task. Spatial Atlas places a spatial handler and an ML-engineering handler behind one A2A server, and the two handlers share a model client, model tiers, and usage tracking.

Spatial reasoning in vision-language models. General-purpose vision-language models (VLMs), such as models trained with visual instruction tuning [24], answer free-form questions about images. VLMs remain weak at quantitative spatial reasoning, such as estimating distances and size differences between objects [6, 22, 7]. One study shows that VLMs often describe objects that are not present in the image [21]. SpatialVLM [6] attempts to address the spatial weakness through specialized spatial training data. Our approach moves selected computations into an explicit representation, and it still depends on the accuracy of entity extraction and geometric reconstruction.

Scene graphs for visual reasoning. Visual Genome [20] and the GQA dataset [17] represent visual scenes as graphs of objects and relationships. Scene graph generation [34] predicts such graphs from images, and Hildebrandt et al. [15] answer visual questions by reasoning over scene graphs. Our scene-graph engine adapts these ideas to industrial scenes, and it adds distance computation and safety-rule checks as explicit operations.

Spatial representations and explicit computations. Farhadloo et al. [10] classify multi-category point sets through spatial arrangements and training strategies that account for differences between place-types. The Atlas-EHR vision proposes a spatial hierarchy for navigating biomedical histories and identifies open research questions for decision support [11]. Taxonomy-aware colocation mining computes frequent proximity relationships across a feature hierarchy [14], and super-colocation mining measures the density of interactions among colocated feature instances [1]. These works define spatial representations and the operations that turn them into analytical outputs. Spatial Atlas applies a related separation between representation and computation when it extracts a scene graph and then computes distances or checks rules.

Physical bounds and incomplete spatial evidence. Sharma et al. [27] use time slicing to tighten bounds on possible rendezvous locations in spatial networks during trajectory gaps. Sharma et al. [28] combine movement bounds with a signal-coverage model to detect abnormal trajectory gaps. These methods distinguish possible movement from directly observed movement through explicit physical and spatial assumptions. Spatial Atlas addresses a different task, but its strict metric bridge also requires declared evidence before it supplies a geometric value to the answering model. Spatial Atlas does not implement these trajectory algorithms, colocation miners, spatial classifiers, or biomedical interfaces.

AutoML and competition-oriented systems. Automated machine learning frameworks such as AutoGluon [9], Auto-Sklearn [12], and AutoKeras [19] aim to automate the end-to-end ML pipeline. Hollmann et al. [16] use a language model to write feature-engineering code for tabular data. Our ML path adds strategy-aware code generation and bounded repair attempts behind a fail-closed execution gate.

A2A protocol and agent interoperability. The A2A protocol from Google [13] defines a standard for communication between agents, so that different agents can work together through a common interface. Our system implements an A2A server with the official `a2a-sdk`, and it exposes both spatial reasoning and ML pipeline capabilities through one task interface.

Information-theoretic reasoning. Active learning [26] studies how to choose the most informative examples to label. Bayesian experimental design [4] chooses experiments that maximize an expected utility, and expected information gain is one standard choice of utility. Active prompting [8] uses model uncertainty to choose which questions receive human-written chain-of-thought examples. Our design proposes an entropy-guided extension of this idea to agent action selection, which would estimate which reasoning step most reduces uncertainty about the final answer. No code in the repository calls that selector, so no result here shows its effect.

3 System Architecture

Spatial Atlas runs as a research agent behind a dual-domain A2A server. The server receives task requests through the protocol and routes each one to a domain handler. Figure 1 shows the overall design.

Domain classification. The domain classifier inspects task metadata and attachment types. The FieldWorkArena adapter recognizes its documented goal shape, and MLE-bench tasks arrive with `tar.gz` attachments that hold competition data and description files. Classification uses four fixed rules and no model call. The implementation has not been benchmarked for routing latency or resource use.

3.1 Shared Infrastructure

Both domain handlers share the components described in this subsection.

LiteLLM wrapper. We use LiteLLM [2] to call models from several providers through one interface, and it records provider-reported usage when the provider supplies it. Provider reports and local estimates are not treated as exact tokenizer-equivalent counts.

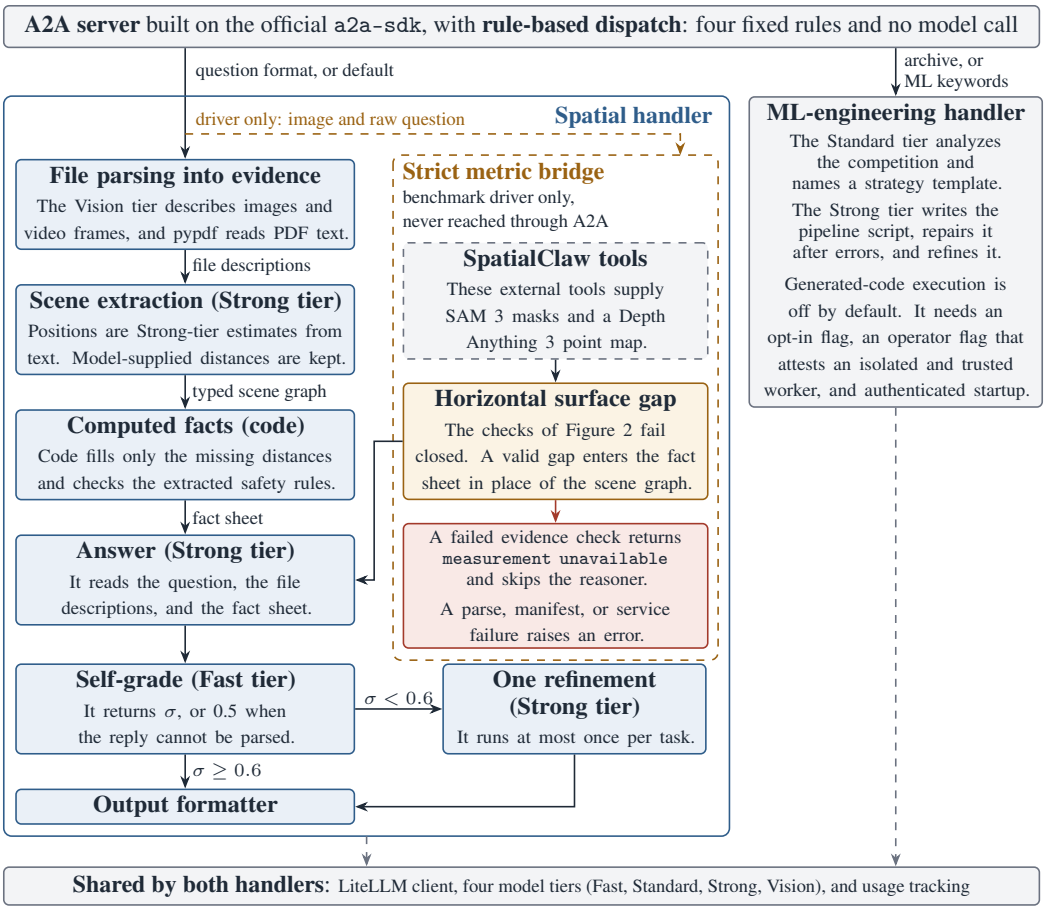

Figure 1: Spatial Atlas places two handlers behind one A2A server. The server passes each task to one handler through four fixed dispatch rules, and no model call takes part in dispatch. On the default spatial path, the Strong tier extracts a typed scene graph from the file descriptions, so entity positions are Strong-tier estimates from text. Code computes only the distances that the model did not supply, and the Strong tier answers from the resulting fact sheet. A Fast-tier self-grade below 0.6 triggers one Strong-tier refinement. The strict metric bridge runs only under the frozen benchmark driver, which supplies the raw question and the protocol identifier. It calls SpatialClaw's tools [7] for SAM 3 masks [3] and Depth Anything 3 point maps [23]. The bridge fails closed and never falls back to the scene graph. The information-gain selector and the tier router exist in code, but no code calls them, so the figure omits them.

Model tiers. We define four model tiers (Fast, Standard, Strong, and Vision), and each tier has a distinct role, as shown in Table 1. Each call site names its tier in code, and no component routes calls between tiers at run time. A tier names a role, and an operator can point the Strong tier at a different provider. Section 5 states which tiers the spatial path invokes and in what order.

The default configuration maps the Standard, Strong, and Vision tiers to GPT-4.1. Operators can override `ATLAS_STRONG_MODEL` to test a different provider. Same-model and cross-provider refinement remain planned ablations. No accuracy or resource-use result is claimed here.

Public agent token reservation. The public A2A `Agent` uses a concurrency-safe client with a per-execution token reservation ceiling, configured by default at 150,000 tokens. That figure is a configured cap, and this paper reports no observed token usage. Before each provider call, the client estimates the prompt size and reserves tokens under a lock, as in Eq. (1).

$$m' = \min\big(m,\ L - U - \mathcal{R} - \hat{p}\big), \qquad \text{refuse the call if } m' \le 0,$$
$$\mathcal{R} \leftarrow \mathcal{R} + \hat{p} + m' \quad \text{otherwise.} \tag{1}$$

Here $L = 150{,}000$ tokens per execution, and U is the committed total. The symbol $\mathcal{R}$ is the outstanding reservation, m is the requested completion limit, and $\hat{p}$ is the prompt estimate. After the

Table 1: This table lists the configured model tiers and their uses in the current code. It is a design table, and it reports no performance or resource-use result.

Tier	Model	Current use
Fast	GPT-4.1-mini	Self-grade, and entity phrases for the generic metric engine
Standard	GPT-4.1	MLE competition analysis
Strong	GPT-4.1	Scene-graph extraction, spatial answers, refinement, code generation, and code repair
Vision	GPT-4.1	Descriptions of images and video frames

call, $\mathcal{R}$ drops by $\hat{p} + m'$ and U rises by the same amount. The counter therefore commits each whole reservation, whatever the provider later reports.

The prompt estimate is $\max(1, \lceil \mathrm{len}/4 \rceil)$ tokens for each text part. An inline image string counts as $2{,}048$ tokens, and an image sent to the vision call counts as $\max\big(1{,}024, \min(8{,}192, \lceil \mathrm{bytes}/1{,}024 \rceil)\big)$ tokens. Concurrent calls within one A2A execution cannot oversubscribe this estimated counter. Every execution builds a fresh agent, so a new execution receives a fresh counter even for the same A2A task ID. The estimate is heuristic because provider tokenizers and image accounting vary, so the ceiling is not a hard provider-token boundary. The frozen benchmark driver uses the base client, which records usage and reserves nothing. It writes each row's usage counts into its journal artifacts.

4 Spatial Scene Graph Engine

The spatial scene-graph engine is the central component of the spatial question-answering path and of the unexecuted FieldWorkArena adapter. It makes spatial computations explicit, but it does not remove errors introduced by perception, depth estimation, object matching, or coordinate assumptions.

4.1 Problem Formulation

The input is an image I of an industrial scene, such as a factory, a warehouse, or a retail space, together with a natural language question q. The task is to produce an answer a, which may require counting objects, estimating distances, checking spatial containment, or verifying safety compliance. Directly prompting a VLM with (I, q) is unreliable, because VLMs can describe objects that are not in the image [21] and remain weak at estimating distances [6].

4.2 Scene Graph Construction

Our approach splits the problem into three stages, which are *extraction*, *structuring*, and *computation*.

Stage 1 (extraction). Entity extraction has up to three steps. When the optional `torch` and `transformers` packages are installed, Florence-2 [33] runs first and returns object counts, keyword-based protective-equipment flags, and a caption. The repository does not declare these packages, so a default install skips this step. Next, the Vision tier (GPT-4.1 by default) writes a detailed description that lists the visible objects with approximate positions and attributes. Its prompt tells it to reuse the Florence-2 counts when they exist. Finally, the Strong tier reads the first $8{,}000$ characters of these descriptions and returns a JSON list of entities, relations, zones, and safety rules. Florence-2 bounding boxes are computed but never passed to a model, and detection quality on industrial imagery has not been measured here.

Stage 2 (structuring). The extracted entities form a spatial scene graph $G = (V, \mathcal{E})$. Its vertices V represent entities, and its edges $\mathcal{E}$ represent spatial relations, as in Eqs. (2) and (3).

$$v_i = \texttt{SpatialEntity}(\mathrm{id}_i, \mathrm{label}_i, \mathrm{pos}_i, \mathrm{attrs}_i, \mathrm{zone}_i), \tag{2}$$

$$e_{ij} = \texttt{SpatialRelation}(\mathrm{subj}_i, \mathrm{pred}_{ij}, \mathrm{obj}_j, d_{ij}). \tag{3}$$

Here $\mathrm{pos}_i \in \mathbb{R}^2$ is a position that the Strong tier estimates from the text descriptions and returns in its JSON extraction. The dictionary attrs_i holds visual attributes such as color, size, and state, and zone_i

names the semantic zone, such as a loading dock or an aisle. The value d_{ij} is the distance attached to the relation. The object of a relation can also name a zone, so an edge need not join two vertices. When the extraction supplies a distance, the graph keeps that model-estimated value. Otherwise the code computes the Euclidean distance between the two positions and rounds it to two decimals, as in Eq. (4).

$$d_{ij} = \begin{cases} \hat{d}_{ij} & \text{if the extraction supplies a distance,} \\ \mathrm{round}_2\big(\|\mathrm{pos}_i - \mathrm{pos}_j\|_2\big) & \text{if both positions exist,} \\ \text{undefined} & \text{otherwise.} \end{cases} \tag{4}$$

Stage 3 (computation). The scene graph also exposes the three operations below. The pipeline first fills missing distances with `compute_all_distances`, as in Eq. (4), and it then calls `check_constraints` and `to_fact_sheet`. No pipeline code calls `query_near`, and only the unit tests exercise it.

- `query_near(`v`, `r`)` returns all entities within radius r of entity v, as in Eq. (5). It recomputes each distance from the stored positions and ignores any stored relation distance.
- `check_constraints()` takes no argument. It reads the safety rules that the Strong tier extracted. It flags a worker whose extracted attributes mark missing protective equipment, and it flags a person-to-hazard relation whose distance is below a threshold parsed from a rule. It returns the list of violations.
- `to_fact_sheet()` serializes the graph into a structured natural-language summary for the language model. It prints a horizontal-gap relation with six decimals and any other distance with one decimal.

$$N_r(v) = \{\, u \neq v \mid \|\mathrm{pos}_u - \mathrm{pos}_v\|_2 \leq r \,\}, \tag{5}$$

$$\mathrm{viol}(e_{ij}) = [\ell_i \in \mathcal{P}] \wedge [\ell_j \in \mathcal{H}] \wedge [d_{ij} < r_{\text{rule}}]. \tag{6}$$

In Eq. (6), the bracket $[\cdot]$ is 1 when its condition holds and 0 otherwise. The symbol ℓ_i is the label of entity i. The person set is $\mathcal{P} = \{\text{worker}, \text{person}, \text{employee}\}$, and the hazard set is $\mathcal{H} = \{\text{forklift}, \text{machinery}, \text{crane}, \text{conveyor}, \text{vehicle}\}$. The labels are lowercased and must match a set member exactly. The threshold r_{rule} is the first number that is followed, after optional spaces, by the letter m in a rule whose text contains "distance", "meters", or "from".

The Strong tier then receives the question, the file descriptions, and the fact sheet. The fact sheet mixes model-estimated positions, model-supplied distances, and code-computed distances, so it does not remove visual estimation from the answer.

4.3 Strict Metric Perception Bridge

The scene graph above does repeatable arithmetic over *model-estimated* two-dimensional coordinates. The arithmetic is deterministic, but its inputs are not measurements. The strict metric bridge replaces those coordinates with reconstructed geometry for one question type. It computes the gap between two objects from segmentation masks and a reconstructed metric point map. Its strict parser targets the horizontal-gap questions of Q-Spatial Bench [22]. Only the frozen benchmark driver reaches the bridge, as Section 4.4 explains. Figure 2 shows its steps. Most constants below belong to a frozen geometry contract, and the code serializes that contract and hashes it with SHA-256. The 0.30 instance-area ratio in the pair-selection step sits outside that contract.

Mask alignment and erosion. The bridge first resizes each object mask to the $w_t \times h_t$ grid of the point map with nearest-neighbor sampling. It then erodes the resized mask $\tilde{M}$ once with a closed Euclidean disk and a zero border, as in Eq. (7).

$$r_e = \left\lceil \max\!\left(\frac{w_m}{w_t}, \frac{h_m}{h_t}\right) \right\rceil, \qquad B_{r_e} = \{(u,v) \in \mathbb{Z}^2 \mid u^2 + v^2 \leq r_e^2\}, \qquad E = \tilde{M} \ominus B_{r_e}. \tag{7}$$

Here (w_m, h_m) is the original mask size, and r_e is the erosion radius in point-map pixels. An empty eroded mask E makes the pair invalid.

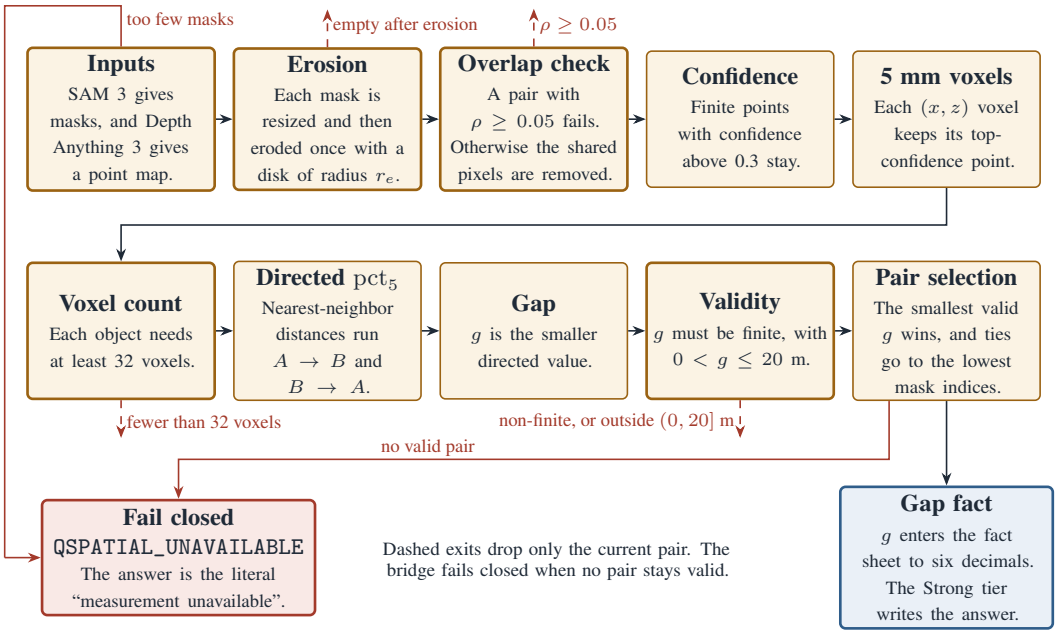

Figure 2: The strict metric bridge computes a horizontal gap from object masks and a point map. Each candidate pair of object masks passes through the steps from left to right, first along the top row and then along the bottom row. Here pct_5 is the fifth percentile with linear interpolation, as in Eq. (12). The erosion radius r_e is the ceiling of the larger resolution ratio between a mask and the point map. Each mask is first resized to the point-map grid by nearest-neighbor sampling. The overlap coefficient is $\rho = |E_a \cap E_b| / \min(|E_a|, |E_b|)$ for the eroded masks E_a and E_b. The bridge returns `QSPATIAL_UNAVAILABLE` at once when the parser marks the question unsupported or when segmentation returns too few masks. A parse or manifest failure raises an error before the steps in this figure begin. Every dashed exit drops only the current pair, and the bridge returns `QSPATIAL_UNAVAILABLE` when no pair stays valid. The confidence filter has no exit of its own, because a mask that loses its points fails the voxel count. A segmentation or reconstruction service failure raises an error, and the bridge never falls back to the scene graph. A valid gap enters the fact sheet, and the Strong tier writes the final answer from it.

Overlap rule. Two eroded masks may still share pixels. The bridge measures their overlap with the overlap coefficient ρ in Eq. (8). A pair whose coefficient is 0.05 or more is rejected. Otherwise the shared pixels are removed from both masks.

$$\rho(E_a, E_b) = \frac{|E_a \cap E_b|}{\min(|E_a|, |E_b|)}, \qquad \begin{cases} \text{reject the pair} & \text{if } \rho \geq 0.05, \\ E_k \leftarrow E_k \setminus (E_a \cap E_b) \text{ for } k \in \{a, b\} & \text{otherwise.} \end{cases} \tag{8}$$

Confidence filter and voxels. A pixel p of an eroded mask contributes only if its reconstructed point $X(p)$ has three finite coordinates and its reconstruction confidence $c(p)$ is finite and exceeds 0.3. The code treats world axes 0 and 2 of the point map as the horizontal plane and axis 1 as the vertical axis. It assumes that the reconstruction is already metric and gravity-aligned, and it does not check either property. The bridge then bins the qualified points on the horizontal x and z axes into $5\,\mathrm{mm}$ voxels with the key κ in Eq. (9). Each occupied voxel keeps one representative point, chosen by the rule π in Eq. (10).

$$Q(E) = \{\, p \in E \mid X(p) \text{ finite},\ c(p) \text{ finite},\ c(p) > 0.3 \,\},$$

$$\kappa(p) = \left(\left\lfloor \frac{X_x(p)}{0.005} \right\rfloor, \left\lfloor \frac{X_z(p)}{0.005} \right\rfloor \right), \tag{9}$$

$$\pi(k) = \operatorname*{arg\,max}_{p \in Q(E),\ \kappa(p) = k} \big(c(p), -\mathrm{idx}(p) \big). \tag{10}$$

Here $\mathrm{idx}(p)$ is the flat pixel index, and point coordinates are in meters. The arg max is lexicographic, so the highest confidence wins and a tie goes to the lowest index. SAM 3 segmentation is requested with a 0.3 threshold, and the contract records that rule as at least 0.30.

Voxel count limits. The set $K(E)$ holds the occupied voxel keys of a mask. A mask with more than 50,000 keys keeps the 50,000 keys with the smallest SHA-256 digests, so the retained subset

does not depend on sampling order. After this cap, each mask must keep at least 32 keys, as in Eq. (11), and a mask with fewer keys makes the pair invalid.

$$K(E) \leftarrow \begin{cases} \text{the } 50{,}000 \text{ keys with the smallest } \mathrm{SHA256}(k_x, k_z) & \text{if } |K(E)| > 50{,}000, \\ K(E) & \text{otherwise,} \end{cases}$$
$$|K(E)| \geq 32. \tag{11}$$

Here $\mathrm{SHA256}(k_x, k_z)$ is the digest of the UTF-8 string of the two signed integers joined by a comma, and digests are compared as bytes.

Surface gap and validity. The sets R_A and R_B hold the x-z coordinates of the voxel representatives of objects A and B. For each representative of one object, the bridge finds its Euclidean nearest neighbor in the other object with a `scipy cKDTree`. It summarizes each directed distance set by its fifth percentile with linear interpolation, and it takes the smaller of the two directed values as the gap g in Eq. (12).

$$\mathrm{nn}_{A\to B}(\mathbf{u}) = \min_{\mathbf{w}\in R_B} \|\mathbf{u} - \mathbf{w}\|_2 \quad (\mathbf{u} \in R_A),$$
$$g = \min\Big(\mathrm{pct}_5\big(\{\mathrm{nn}_{A\to B}(\mathbf{u})\}_{\mathbf{u}\in R_A}\big),\ \mathrm{pct}_5\big(\{\mathrm{nn}_{B\to A}(\mathbf{w})\}_{\mathbf{w}\in R_B}\big)\Big),$$
$$0 < g \leq 20 \text{ m}. \tag{12}$$

Here pct_α is the linear-interpolation percentile at the zero-based position $(\alpha/100)(n-1)$ among n values sorted in ascending order. A gap that is not finite or lies outside the interval $(0, 20]$ m makes the pair invalid.

The fifth percentile limits the influence of a few extreme distances while keeping the near-contact region that the question asks about. At the 32-voxel minimum it falls between the second and third smallest distances, so the single smallest distance in each direction does not set the gap. A stray reconstructed point close to the other object can still lower the gap, and its effect on gap error has not been evaluated. Taking the symmetric minimum removes the dependence on which object is named first.

Instance filter and pair selection. A distinct-pair question names two different objects, and every cross pair of their instances is a candidate. A repeated-instance question asks about two instances of the same object. For such a question, an instance enters pairing only if its mask area is at least 0.30 of the largest instance area, and at least two instances must remain. Every pair of the remaining instances is a candidate. The bridge selects the valid pair with the smallest gap and breaks ties by the lowest mask indices, as in Eq. (13).

$$\mathcal{I} = \{\, i \mid \max_j \mathrm{area}_j > 0,\ \mathrm{area}_i \geq 0.30 \max_j \mathrm{area}_j \,\}, \qquad (i^*, j^*) = \operatorname*{arg\,min}_{(i,j)\ \text{valid}} \big(g_{ij},\ (i,j)\big). \tag{13}$$

Here area_i is the pixel count of the raw segmentation mask of instance i, and $\mathcal{I}$ applies only to repeated-instance questions. The arg min is lexicographic. The 0.30 instance-area ratio was chosen by inspecting development cases, and its effect on gap error has not been evaluated. Because the ratio sits outside the hashed contract, a change to it would leave the contract digest unchanged.

Output and failure behavior. Equation (14) gives the outcome B of the strict bridge.

$$B = \begin{cases} g_{i^*j^*} \text{ in meters, in the fact sheet} & \text{if a valid pair exists,} \\ \texttt{QSPATIAL_UNAVAILABLE} & \text{if an evidence check fails,} \\ \text{an error} & \text{if parsing, the manifest check, or a service fails.} \end{cases} \tag{14}$$

The evidence checks are an unsupported question, an empty segmentation, fewer than two plausible instances, and the absence of any valid pair. The fact sheet prints g to six decimals. The constant `QSPATIAL_UNAVAILABLE` holds the literal answer `measurement unavailable`, and it becomes the final answer. After a valid measurement, the handler passes the fact sheet to the controller in Section 5.1. The Strong tier then writes the final answer and may refine it once, so the gap reaches the answer only through the model. No code checks that the final answer repeats g.

Two properties of this path matter for interpretation. First, it is deliberately *fail-closed*. It records protocol identifiers, provenance, evidence digests, and range checks. When an evidence check

fails, it returns the literal answer `measurement unavailable` before the reasoner runs. It raises an error when the question falls outside the parser grammar, when the sample metadata fail their manifest check, or when the segmentation or reconstruction service fails. It never substitutes the estimated-coordinate scene graph in either case. Second, reconstructed geometry is an estimate and not ground truth. Segmentation errors, reconstruction scale errors, and referent mismatches all remain possible. The fail-closed checks catch only the failures that break an evidence check. A wrong mask or a wrong scale that passes every check still produces a wrong gap.

The strict parser was written against Q-Spatial Bench question text. It carries hand-written aliases for specific phrasings, it marks three rows as unsupported, and a separate table rewrites some grounding queries sent to SAM 3. That table sits outside the hashed contract. The parser is therefore fitted to this benchmark and is not a held-out component.

The bridge code is public, but it is not a stand-alone reproduction. The live perception path imports its segmentation and reconstruction tools from an externally installed copy of SpatialClaw [7]. SpatialClaw is a separate training-free framework that uses code as its action interface in a persistent kernel. Its GPU service runs SAM 3 segmentation [3] and Depth Anything 3 reconstruction [23]. Neither SpatialClaw nor its GPU service is vendored here, and the gated benchmark images and the frozen control artifact are not bundled with the repository.

4.4 Generic Metric Engine

The scene-graph engine is the default, and an operator selects the metric engine through configuration. On the A2A path, metric-engine requests use a generic metric engine. The A2A server never passes the strict protocol identifier, so an A2A request in metric mode always uses the generic engine. Only the frozen benchmark driver reaches the strict bridge. The driver can also pass exact benchmark regions, and that path uses SpatialClaw's centroid rule without a Fast-tier call. In the generic engine, the Fast tier lists up to six entity phrases to segment. For each object i, the engine keeps the mask points Q_i whose coordinates are finite and whose confidence exceeds 0.3. It then computes a position and a visible height, as in Eq. (15).

$$\begin{aligned} \mathrm{pos}_i &= \big(\mathrm{med}_{p\in Q_i}\, X_x(p),\ \mathrm{med}_{p\in Q_i}\, X_z(p)\big), \\ h_i &= \mathrm{pct}_{97.5}\big(\{X_y(p)\}_{p\in Q_i}\big) - \mathrm{pct}_{2.5}\big(\{X_y(p)\}_{p\in Q_i}\big), \qquad |Q_i| \geq 32,\ \ h_i > 0. \end{aligned} \tag{15}$$

The centroid is the median of the confident reconstructed points, and the engine stores its x and z coordinates as the entity position in the scene graph. Visible height is the 97.5th percentile minus the 2.5th percentile of the vertical coordinate. A height needs at least 32 qualified points, and a height of zero or less is rejected.

The two metric paths fail in different ways. When strict mode is off, which is the A2A default, the generic engine falls back to the scene-graph path after a failure in the metric backend. In that mode, a failed instance is skipped, and a metric scene with no entities is kept rather than replaced. The strict path never falls back, and it fails closed as described in Section 4.3. The frozen benchmark driver turns on strict mode whenever the metric engine is selected.

4.5 Label-Free Run Modes and Validators

The repository provides the run modes and validators for a four-path comparison. Two options select a path. The engine option is the scene-graph engine or the metric engine, and the image-mode option is the correct image, the question only, or a shuffled image. The options are not freely combinable, because the driver enforces four constraints. The question-only mode requires the scene-graph engine, because it withholds the image and so has no geometry to measure. The shuffled-image mode requires the metric engine and an explicit control mapping, and a control mapping is rejected for any other image mode. The driver therefore rejects the pairs (metric, question-only) and (scenegraph, shuffled). Only four of the six combinations are admitted, as Eq. (16) states.

$$\begin{aligned} \mathcal{M} = \big\{&(\text{scenegraph}, \text{correct}),\ (\text{scenegraph}, \text{question-only}), \\ &(\text{metric}, \text{correct}),\ (\text{metric}, \text{shuffled})\big\}. \end{aligned} \tag{16}$$

A control mapping is required exactly when the image mode is shuffled. The four admitted paths are the scene graph with the correct image, the question-only baseline, the metric engine with the correct image, and the metric engine with a shuffled image.

The shuffled-image control is bound to a contract. The control mapping ϕ is defined over the set S of source image paths, and δ_s is the recorded SHA-256 digest of the image stored at path s. The loader checks the size of the mapping, its bijectivity, the absence of fixed points, and the digest of every source and control image, as Eq. (17) states.

$$\begin{aligned} &\phi : S \to S, \qquad |S| = 84, \qquad \phi \text{ is a bijection}, \qquad \phi(s) \neq s \ \ \forall s \in S, \\ &\mathrm{SHA256}(s) = \delta_s \ \text{ and } \ \mathrm{SHA256}\big(\phi(s)\big) = \delta_{\phi(s)} \ \ \forall s \in S. \end{aligned} \tag{17}$$

Here $\mathrm{SHA256}(s)$ is the digest of the image stored at path s. The loader rejects the run if any image has drifted or if a source repeats. The mapping therefore holds exactly 84 pairs, and no source path maps to itself. The loader compares paths and does not require $\delta_s \neq \delta_{\phi(s)}$, so it would admit two byte-identical images stored under different paths. The mapping metadata must also name a fixed cyclic-next algorithm over byte-sorted paths.

Prediction rows are appended to journals under the label-free schema `label_free_prescore_v1` with an append-only file flag. The writer rejects label-bearing keys anywhere in a payload, and it uses an exact-key list plus token and prefix rules to find them. Prediction metadata must contain exactly six safe fields. The writer also refuses to mix label-free and ordinary rows in one journal, and it refuses to resume a journal whose schema does not match the current run.

These validators keep label-bearing fields out of the prescore journals and the recorded metric evidence. They check what a run writes, so the claim that labels stayed sealed in the reported run rests on the run record in Section 8. The public repository does not provide the combined single-command orchestrator that runs all four paths under one root.

Output formatting against benchmark scorers. FieldWorkArena scoring is done by the benchmark's own evaluation program [30], and this repository implements none of its scoring functions. It does implement an output formatter that shapes each answer, so that a correct answer is not rejected on presentation. A right answer in the wrong shape can score zero under exact and substring matchers. The public FieldWorkArena evaluator uses string rules for its exact, include, and exclude matchers, and it asks a language model to grade its fuzzy, JSON, and numerical matchers [30].[1] Because the gated benchmark data were never accessible, this formatter has never been exercised against an official FieldWorkArena evaluation run.

5 Confidence-Gated Refinement and a Proposed Entropy-Guided Design

This section separates the controller that the spatial path executes from an information-theoretic design that no code executes. The design draws on active learning [26] and Bayesian experimental design [4]. The difference between the two parts matters for how the design should be judged.

5.1 Implemented Controller

The implemented controller answers once at the Strong tier, asks the Fast tier for one self-grade, and refines at most once, as in Eqs. (18) to (20).

$$a_0 = \mathrm{Strong}\big(q,\, D_{:12000},\, F\big), \tag{18}$$

$$\sigma = \begin{cases} \mathrm{parse}\big(\mathrm{Fast}(q,\, D_{:2000},\, a_0)\big) & \text{if the reply parses,} \\ 0.5 & \text{otherwise,} \end{cases} \tag{19}$$

$$a = \begin{cases} \mathrm{Strong}\big(q,\, a_0,\, D_{:6000},\, F\big) & \text{if } n_{\text{refl}} > 0 \text{ and } \sigma < 0.6, \\ a_0 & \text{otherwise.} \end{cases} \tag{20}$$

Here $D_{:n}$ denotes the first n characters of the file descriptions, and F is the fact sheet. Both Strong-tier prompts also carry the expected output format, and F is empty when the scene has no entities. The value n_{refl} is the reflection setting, whose default is 2 and which acts only as an on-off flag. The self-grade runs only when $n_{\text{refl}} > 0$, and Algorithm 1 restates the same procedure.

The implemented controller asks the Fast tier for one self-reported score σ for the single Strong-tier answer. It does not approximate the information-gain rule that Section 5.2 proposes. The prompt

[1] The evaluator code is in the public repository at https://github.com/FujitsuResearch/FieldWorkArena.

Algorithm 1 This procedure is the implemented spatial reasoning controller, with $D_{:n}$ as defined above. The self-grade σ is a self-reported heuristic. The code does not clip it, and it is not calibrated.

Require: Question q, file descriptions D, fact sheet F, threshold $\tau = 0.6$
$a \leftarrow \textsc{Strong}(q, D_{:12000}, F)$
if reflection is disabled **then**
    return a
end if
$\sigma \leftarrow \textsc{Fast-SelfGrade}(q, D_{:2000}, a)$, or $\sigma \leftarrow 0.5$ when the reply cannot be parsed
if $\sigma < \tau$ **then**
    $a \leftarrow \textsc{Strong-Refine}(q, a, D_{:6000}, F)$ ▷ runs at most once
end if
return a

requests a value between 0.0 and 1.0, but the code does not clip the parsed value, and a missing or unparseable score defaults to 0.5. This score has not been shown to be calibrated, so it should not be read as a posterior probability.

During reflection, the Strong tier receives the question, its first answer, the first 6,000 characters of the file descriptions, and the fact sheet, and it writes one revised answer. The first answer had already seen the first 12,000 characters of the same descriptions, so reflection adds no new evidence. Because the 0.5 default lies below $\tau = 0.6$, an unparseable self-grade always triggers this refinement. The configuration field named for reflection rounds acts as an enable flag, so the controller performs at most one refinement per task and never loops.

5.2 Proposed Entropy-Guided Design

The formulation in this subsection is a proposed design. No code in the repository computes Eq. (21) or runs the selection rule in Eq. (22).

At each reasoning step t, an agent under this design would keep a knowledge state $\mathcal{K}_t$ of accumulated observations, computed facts, and intermediate conclusions. The *answer entropy* in Eq. (21) measures the uncertainty over the space of possible answers.

$$H(\mathcal{A} \mid \mathcal{K}_t) = -\sum_{y \in \mathcal{A}} P(y \mid \mathcal{K}_t) \log P(y \mid \mathcal{K}_t). \tag{21}$$

Here $\mathcal{A}$ is the set of candidate answers, and $P(y \mid \mathcal{K}_t)$ is the estimated probability of answer y given the current knowledge. The design would consider n_c candidate actions $\{c_1, \ldots, c_{n_c}\}$, such as examining one region of the image, querying the scene graph, or calling a stronger model. It would then select the action with the largest expected information gain, as in Eq. (22).

$$c^* = \arg\max_{c_j} \; \mathbb{E}_{o \sim P(o \mid \mathcal{K}_t, c_j)} \big[H(\mathcal{A} \mid \mathcal{K}_t) - H(\mathcal{A} \mid \mathcal{K}_t \cup \{o\}) \big]. \tag{22}$$

Here o is the observation that action c_j would return. The repository defines a selector for this rule, and that selector would ask the Fast tier to rate each candidate action from 1 to 10. No code in the repository calls it, so this paper makes no claim that the selector runs or reduces model calls.

5.3 Model Routing

A graded routing policy is the design target for efficient use of model calls. Under it, a fast tier would answer confidently handled questions, a standard tier would handle moderate cases, and a strong tier would take the hardest ones. We state this policy as a target, and it does not describe the current spatial path.

The repository also defines a tier-router class, and no code in the repository calls it. The implemented spatial controller answers directly at the Strong tier over the evidence already built, uses the Fast tier only for the self-grade, and escalates no further than one Strong-tier refinement. It does not perform graded escalation. So this paper makes no efficiency claim for the spatial path. Whether graded routing reduces model usage without degrading answers is an open question for the planned evaluation, and answering it requires the protected scoring that has not yet been performed. The effect of the routing policy on resource use is likewise unmeasured.

Table 2: This table lists the strategy templates that the handler inserts into the code-generation prompt. The Strong tier writes the final script, so a template guides the code and is not the executed code.

Strategy name	Task type	Template contents
`tabular`	Tabular classification or regression	AutoGluon `TabularPredictor` with a 300-second limit, which falls back to LightGBM when AutoGluon is not installed
`nlp`	Text classification	TF-IDF unigram and bigram features (up to 50,000) with logistic regression and a cross-validation check
`vision`	Image classification	Flattened 32×32 pixel features with a tree-ensemble classifier
`timeseries`	Forecasting	Lag features (1, 7, 14, and 28 steps), rolling features, and a LightGBM regressor
`general`	Mixed or unknown	A gradient-boosting classifier or regressor

6 Fail-Closed ML Pipeline

The ML-engineering handler generates one pipeline script from each competition description. Execution is fail-closed, and it requires both `ATLAS_ENABLE_MLEBENCH_CODE_EXECUTION=true` and `ATLAS_TRUSTED_ISOLATED_WORKER=true`. The second flag is an operator's attestation that the process runs in an isolated, trusted worker, and the code cannot confirm that isolation. Server startup then requires an `ATLAS_BEARER_TOKEN` with at least 32 characters.

Competition analysis. When a competition task arrives, the Standard tier reads the description, a file listing, and a data preview. It returns JSON with a task type, a metric and its direction, a target column, the submission format, a short data summary, a strategy name, and key insights. The strategy name selects a template from Table 2.

6.1 Code Generation and Execution

For each competition, the pipeline asks the Strong tier for a single standalone Python script with the steps below.

1. The script loads and preprocesses the training data according to the detected task type.
2. It implements the selected strategy with suitable hyperparameters.
3. It holds out a simple validation split, trains the model, and prints one `VALIDATION_SCORE` line for that split.
4. It generates predictions on the test set in the required submission format.
5. It writes a valid `submission.csv` to the expected output location.

After explicit authorization, the generated script runs in a bounded subprocess with a 600-second timeout. The subprocess captures bounded stdout and stderr and uses a minimal environment. It is terminated as a process group on timeout or when the handler cancels the run internally. The A2A cancel operation itself is not supported. These controls are defense in depth, and they do not form a complete security sandbox.

Bounded repair loop. When an explicitly authorized execution fails, the handler runs the repair steps below.

1. **Error capture.** The executor keeps the tail of stderr and stdout from the failed run.
2. **Repair.** The Strong tier receives the script, the last 2,000 characters of the error, the last 1,000 characters of stdout, and the competition description, and it returns a complete revised script. Its prompt asks it to change only what the error requires.
3. **Re-execution.** The handler runs the revised script under the same timeout.

The setting `max_code_iterations = 3` allows 3 total attempts, including the initial attempt. If all attempts fail, the task fails by default. A schema-shaped dummy submission is produced only when the operator separately sets `ATLAS_ALLOW_DUMMY_SUBMISSION=true`.

Score-driven refinement loop. Error recovery can repair a pipeline that crashes, but it cannot raise the score of a working pipeline. After the first authorized run succeeds, the handler parses the last match of the pattern `VALIDATION_SCORE: <float>` in the script's standard output, and it skips refinement when the first run prints no score. Otherwise it requests one targeted revision, runs it under the same authorization and controls, and keeps it only when the parsed score is better under the metric direction. Equation (23) states the attempt limits and this acceptance rule, with ν as the best parsed score so far and ν' as the revised score.

$$n_{\text{attempt}} \le 3, \qquad n_{\text{refine}} \le 2, \qquad t_{\text{run}} \le 600 \text{ s},$$
$$\text{accept } \nu' \iff \begin{cases} \nu' < \nu & \text{if the direction text contains a minimize keyword,} \\ \nu' > \nu & \text{otherwise.} \end{cases} \tag{23}$$

Here n_{attempt} counts the initial run and its repairs, n_{refine} counts the refinement runs, and t_{run} is the timeout of each run. The minimize keywords are `min`, `lower`, `loss`, `error`, `rmse`, `mae`, and `mse`. The loop runs up to `max_refinement_iterations = 2` extra passes, and it starts no new pass once `refinement_wall_time_seconds = 900` have elapsed since the first successful run. The 900-second check runs before each pass, so a pass that starts before that point can finish after it. The selection logic discards revisions that regress or fail to print a score.

This loop uses the configured Strong tier. The Strong tier defaults to GPT-4.1, and the operator can override it. Whether a cross-provider override outperforms a same-model retry is a planned ablation and not an established result.

6.2 Leak Audit and Leak Hint Registry

A competition can leak test information through training-set overlap, public dataset ancestry, or file metadata. Spatial Atlas keeps a *leak hint registry* whose entries are text instructions injected into the Strong-tier code-generation prompt when a competition is detected. It does not hand-code exploit solvers, because their fixed merge keys may not match the MLE-bench tar layout.

Every code-generation call also receives a universal *leak audit preamble*. The preamble asks the Strong model to run four checks before it trains any model.

1. The model compares ID-like columns between train and test for row-level overlap.
2. It computes row fingerprints, which hash the non-target features, to detect duplicated content.
3. It checks temporal ordering in timestamp-based competitions, where temporal shuffling can leak test information.
4. It hashes file bytes in media-based competitions to detect identical train and test files.

The audit text also tells the model to use the train labels for the matching rows when more than 50% of rows overlap. The audit runs whether or not a registered entry matches. Its detection rate has not been measured, so this paper makes no claim that it finds any leak.

The registry holds one entry, for the Random Acts of Pizza competition. When its detection predicate matches, a targeted hint is appended after the generic audit text. The Strong model writes the final pandas operations against the actual tar layout it sees at run time, and the audit policy stays in a single file (`mlebench/strategies/leaks.py`).

Strategy selection. The Standard-tier analyzer picks one strategy template from the task description. The ML path does not call the entropy module (`src/entropy/engine.py`), and it does not generate candidate solutions for several strategies. Entropy-guided strategy selection is not implemented.

7 Implementation Details

A2A protocol. Spatial Atlas implements the A2A protocol with the official `a2a-sdk`. The project requires version 0.3.20 or later, and its lock file pins version 0.3.25, which implements A2A specification v0.3.0 [13]. The server exposes an A2A endpoint that accepts JSON-RPC task submissions and returns structured results in the protocol format. It sends intermediate status updates through the SDK's task updater, and the agent card advertises streaming. The agent card also describes both FieldWorkArena and MLE-bench task types.

Deployment. The system is packaged as a Docker image built from the repository's Dockerfile. Environment variables configure API keys, model endpoints, and resource limits. Public HTTP request bodies default to a 64 MiB limit. At most four non-read-only requests run at once by default, and the server answers HTTP 503 beyond that limit instead of queueing without bound. A configured `ATLAS_BEARER_TOKEN` must contain at least 32 characters, and it authenticates non-read-only requests. Normal non-loopback startup requires the token even when MLE execution is disabled, and loopback development may omit it. The setting `ATLAS_ALLOW_UNAUTHENTICATED_PUBLIC=true` is a test-only override for non-loopback binding. MLE execution always requires authenticated startup.

File processing pipeline. Task inputs arrive in several formats, and each format has its own processing path.

- **Images.** When Florence-2 is available, it runs first and supplies object counts and a caption. The Vision tier (GPT-4.1 by default) then describes the scene. Images in RGBA, LA, or palette mode are converted to RGB, and every image is re-encoded as JPEG before the call. No resizing is applied.
- **PDFs.** Text is extracted page by page with `pypdf`. The pipeline has no OCR step, so a scanned page without a text layer yields no text.
- **Videos.** OpenCV samples one frame every two seconds, and a long video is sampled more sparsely so that at most 30 frames are kept. At most 10 evenly spaced frames are then described. No scene-change keyframe detection is performed.
- **Archives.** The handler extracts `tar.gz` files, which carry MLE-bench data, to a temporary workspace directory.
- **Text.** Files are decoded as UTF-8, and Latin-1 is used as a fixed fallback when UTF-8 decoding fails.

Model configuration. All model calls use the tiers in Table 1. The Fast tier defaults to `openai/gpt-4.1-mini`. It produces the self-grade, and in the generic metric engine it also lists the entity phrases to segment. The Strong tier extracts the scene graph from the file descriptions, and the Vision tier (`openai/gpt-4.1`) describes images and video frames. Domain classification uses fixed rules and no model call. The Standard and Strong tiers both default to `openai/gpt-4.1`, and the Strong tier can be overridden with `ATLAS_STRONG_MODEL`. Any cross-provider variant must be compared with the default under identical token and time limits.

Resource controls. Public A2A model calls use the concurrency-safe per-execution reservation in Eq. (1). The frozen benchmark driver uses the base client, which records usage and reserves nothing. Reflection performs at most one refinement per task. Generated-code execution is disabled by default, and it requires both execution flags plus authenticated server startup. After authorization, each pipeline attempt has a 600-second timeout, and the initial run plus the repair loop permits at most 3 total attempts. Dummy submissions remain disabled unless separately enabled. After a successful real run, the score-driven refinement loop may run up to 2 more passes (Eq. (23)), and it starts no new pass after 900 seconds. Every number in this paragraph is a configured ceiling that bounds worst-case work. None is an observed runtime, an observed attempt count, or a measured resource use.

8 Evaluation

8.1 Current Evidence Boundary

This version reports no claim-bearing benchmark table. FieldWorkArena remained gated and inaccessible, so the project did not run its validation set and reports no FieldWorkArena accuracy, ablation, latency, token, or resource-use result. The local adapter and tests establish software behavior only, and they are not benchmark evidence.

The repository also does not contain a sealed, end-to-end MLE-bench result artifact covering the full competition suite. This version removes the previously drafted valid-submission, medal, refinement, leak-effectiveness, and resource-use figures. A completed job or a working code path is not treated as a scientific result without the corresponding immutable predictions, scorer output, run manifest, and logs.

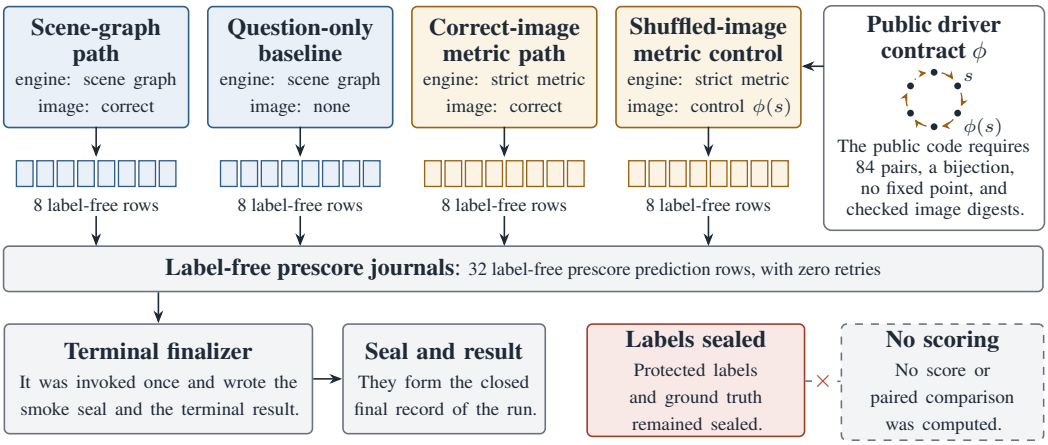

Figure 3: Four operational paths ran together in the V37 run. The paths were the scene-graph path, question-only baseline, correct-image metric path, and schema-matched shuffled-image metric control. Each path produced eight label-free prescore prediction rows, so the run wrote 32 rows with zero retries. The execution, warning-scan, restoration, cleanup, and terminal-seal gates passed. In the public driver, the shuffled-image mode swaps each attached image through a control mapping ϕ. That mapping must hold 84 pairs, be a bijection, and have no fixed point, and the loader checks the digest of every source and control image. The terminal finalizer ran once and wrote the smoke seal and the terminal result. Protected labels and ground truth remained sealed, and no score or paired comparison was computed, so the run establishes operational integrity only. The run is private, and the public repository cannot reproduce it.

The evidence in this paper has two parts, and neither part measures answer quality. The first part is one private label-free operational run, called V37, in Section 8.2. The second part is a pair of software test results in Section 8.3.

8.2 Label-Free Operational Run (V37)

We report one completed label-free operational run of the four-path design. It is an integrity check on the execution path, and it does not measure answer quality. Figure 3 summarizes the four arms of the run.

Four paths ran together. They were the scene-graph path, the question-only baseline, the correct-image metric path, and the schema-matched shuffled-image metric control. Each path wrote eight label-free prediction rows, for 32 rows in total. The run used one submission with zero retries and no rollback. Every bounded execution step exited cleanly, and all eight bounded service logs passed the required warning scan. Registry restoration passed, and the job-local authentication artifact was absent after teardown. The terminal finalizer ran exactly once. It created the terminal result and the smoke seal, which is the closed final record of this smoke-test run.

The shuffled arm is the record's schema-matched shuffled-image metric control. In the public driver, a shuffled-image run is admitted only under the control-mapping contract in Eq. (17). The V37 record does not restate the mapping that the private run loaded, so this paper does not report its size. In the public driver's shuffled mode, the image files sent to the model come from the mapped control images. Whether the strict geometry step in the V37 shuffled arm measured the control image or the source image depends on the private benchmark loader. The public code does not settle that question, so we treat the shuffled arm as an executed control path. We do not treat it as a geometry control.

Table 3 lists what V37 establishes and what it does not. V37 establishes operational integrity only. Protected labels and ground truth were never opened, and no benchmark score, paired comparison, uncertainty interval, resource-use result, or latency result was computed. The shuffled-image path is a control-path *execution*. Because the protected labels stayed sealed, its score and its paired effect against the correct-image path remain unknown.

Three distinctions matter for reading this result. First, the prediction journals record model outputs before scoring. They are not answer-level evidence-use journals, and they do not show which evidence a given answer used. Second, the terminal finalizer checks lifecycle and evidence-integrity conditions. It is not an answer verifier, and it does not check answer correctness. Third, the metric arms used

Table 3: This table quotes the V37 evidence record, which asks writers to use this wording and no stronger interpretation. In item 9 on the right, this paper writes "resource-use" in place of the record's term.

V37 establishes	**V37 does not establish**
1. Four operational paths ran together.	1. Accuracy.
2. The paths were the scene-graph path, question-only baseline, correct-image metric path, and schema-matched shuffled-image metric control.	2. Superiority over another path or system.
3. Each path produced eight label-free prescore prediction rows.	3. A causal or ablation effect.
4. The total was 32 rows.	4. Statistical significance.
5. The execution, warning-scan, restoration, cleanup, and terminal-seal gates passed.	5. An uncertainty interval.
6. The run used zero retries.	6. Calibration.
7. Protected labels and ground truth remained sealed.	7. Benchmark ranking.
8. No score or paired comparison was computed.	8. Production readiness.
9. This establishes operational integrity only.	9. A resource-use or latency result.
	10. Native SpatialClaw persistent-kernel integration.

SpatialClaw perception primitives through the bridge in Section 4.3, and V37 did not run a native SpatialClaw persistent-kernel arm.

The public repository supplies the metric bridge, the four run modes, the control-mapping contract, and the label-free journal validators. The combined four-path orchestration that produced V37 ran in a separate private execution environment, and it is not part of the public repository. Every arm also needs a SpatialClaw checkout and the benchmark data, so the public repository cannot reproduce V37.

8.3 Software Test Results

We report two software test results, and neither one measures answer quality. First, the frozen V37 control suite passed 350 tests plus 63 subtests. The evidence record treats this count as a local regression pass, and it is not benchmark accuracy. Second, a local run of the public CI test command with Python 3.13 reported 323 passed and 1 deselected. The deselected case is the end-to-end test, which needs a GPU tool server and the dataset.

The same CI command reports 92.09% line coverage of `src/fieldwork/perception.py`, and this figure covers that one module only. CI fails when that coverage drops below 90%. The two test counts come from different suites, so they should not be compared with each other. Passing tests establish only the software behavior they exercise, and they do not promote proposed behavior into a result.

9 Planned Evaluation Protocol (Future Work)

The evaluation in this section is planned future work, and none of it has been performed. The spatial evaluation will compare a question-only baseline, the scene-graph path, the metric-perception path, and a native reference implementation on a frozen public slice. It will report per-question-type accuracy, paired uncertainty intervals, parser and geometry failure rates, model and data revisions, token usage, wall-clock latency, and artifact paths. Labels will stay sealed until all prediction journals are complete. Protected scoring, paired analysis, and uncertainty analysis are the next scientific phase.

The protected scoring protocol is specified in advance as the ordered steps below.

1. The analysis plan and the interpretation boundary will be frozen before anything is unsealed.
2. Every prediction journal will be checked for completeness and schema validity.
3. The protected labels will be unsealed only after that check passes.
4. Each path will be scored with the benchmark's own metric.
5. The correct-image metric path will be compared with the scene-graph path and with the shuffled-image control, and each paired comparison will carry an uncertainty interval.
6. Parser failures, geometry failures, refusals, and invalid cases will be reported alongside every aggregate.

The ML-engineering evaluation will run fixed competition subsets with identical token and time limits and report valid-submission rate, competition-specific score, refinement acceptance rate, execution failures, tokens, and latency. Each aggregate must be generated from machine-readable run artifacts, and no aggregate may be copied into the paper by hand.

10 Discussion

Limitations. The list below states the known limits of this work.

- **Estimated coordinates in the default path.** The default scene graph does repeatable arithmetic over model-estimated two-dimensional coordinates. The arithmetic is deterministic, but its inputs are not measured, and deterministic arithmetic cannot correct a wrong geometric input.
- **Benchmark-fitted parser.** The strict parser was written against Q-Spatial Bench question text, with hand-written aliases, three unsupported rows, and a grounding-query override table. It is therefore not a held-out component.
- **Tuned thresholds.** The 0.30 instance-area ratio was chosen by inspecting development cases. Neither its effect on gap error nor the effect of stray reconstructed points on the gap has been evaluated.
- **External perception dependencies.** The metric path requires an externally installed SpatialClaw package and a reachable GPU perception service. Neither one is vendored, so the repository alone cannot run that path.
- **Unbundled data and controls.** The gated benchmark images and the frozen shuffled-image control artifact are not distributed with the repository.
- **No combined orchestrator.** Public source provides the four run modes and their validators, and it does not include the single-root orchestration that produced V37.
- **Inactive information-gain controller.** Expected-information-gain action selection is formulated here and present as a component, but no code in the repository calls it.
- **Uncalibrated and unbounded self-grade.** The self-grade that gates refinement can be any float, and a parse failure defaults to 0.5 and forces the refinement. It has not been shown to be calibrated and should not be read as a probability.
- **Optional detector.** Florence-2 runs only when its optional packages are installed, so a default install skips it.
- **Unvalidated FieldWorkArena adapter.** The adapter was never run against the benchmark, whose data remained inaccessible, so it must not be presented as evaluated compatibility.
- **Proxy validation score.** The ML path parses a validation score that the generated code prints. That score is a proxy supplied by the pipeline under test, and no outside check verifies it as a benchmark metric.
- **Incomplete executor isolation.** The execution controls are defense in depth, and they do not form a complete security sandbox.
- **Hand-designed templates and narrow leak coverage.** The strategy templates target common competition types and may not cover new tasks, and the leak audit covers only four leakage shapes.
- **No cancellation.** The A2A cancel operation is not supported once work is under way.
- **No native persistent loop and no answer-level verification.** The system has no native integration of the SpatialClaw persistent kernel. It also has no journal that records which evidence each final answer used, and no verifier that checks an answer against captured evidence.
- **No scientific comparison.** No protected score, paired arm comparison, uncertainty interval, resource-use result, or latency result has been produced. Accuracy, superiority, calibration, generalization, and production readiness are all unestablished.

Future work. We see five directions for future work.

- **Domain-specific fine-tuning.** Fine-tuning Florence-2 on industrial imagery may improve object detection for domain-specific objects such as safety equipment, pallet types, and industrial signage.
- **Multi-agent collaboration.** The A2A protocol allows architectures in which specialized sub-agents handle sub-tasks, such as visual analysis, spatial computation, and language generation.
- **Richer progress reporting.** The server already streams task status updates. Streaming the training logs of generated pipelines would give finer feedback during long runs.
- **Evidence integration.** Native SpatialClaw persistent-kernel integration, an answer-level evidence-use journal, and a verifier that checks answer claims against captured evidence all remain proposed.

- **Expanded benchmarks.** Extending the architecture to other agent benchmarks, such as SWE-bench [18], would test how general the design is.

Broader impact. The spatial scene graph approach could apply to industrial safety, where automated checks of clearance distances, equipment placement, and emergency exit access could help prevent workplace injuries. Automated spatial reasoning systems must be deployed with human oversight, because errors in safety-critical applications can have severe consequences.

11 Conclusion

We have presented Spatial Atlas, a research agent built on compute-grounded reasoning (CGR) and exposed through an A2A protocol server. Its implemented and proposed parts are listed below.

1. The **spatial scene-graph engine** makes extracted entities, coordinate assumptions, and computed relationships inspectable, and it does not claim that deterministic computation removes perception errors.
2. The **confidence-gated refinement** lets one Fast-tier self-grade trigger at most one Strong-tier refinement. The entropy-guided information-gain selector is a proposed design. No code in the repository calls it, and this paper reports no ablation of its effect.
3. The **fail-closed ML pipeline** combines strategy-aware code generation, explicit execution authorization, bounded repair attempts, and a separately gated dummy fallback.
4. The **score-driven refinement loop** parses validation scores, requests a revision from the Strong tier, and keeps the revision only when its parsed score improves.
5. The **leak audit and hint registry** prompts generated pipelines to check four common leakage patterns, and its one entry adds a targeted hint for a single competition.
6. The **strict metric perception bridge** computes horizontal gaps from segmentation masks and a reconstructed point map, and it passes each gap to the model as a fact. It records provenance and evidence digests, and it fails closed when an evidence check fails.
7. The **label-free run modes and validators** support a four-path comparison. They include a digest-bound, bijective, fixed-point-free shuffled-image control mapping and append-only journals that reject label-bearing fields.

We also report one completed label-free operational run, V37, in which all four paths ran and the execution, warning-scan, restoration, cleanup, and terminal-seal gates passed. Each path produced eight prediction rows, for 32 rows in total, and protected labels and ground truth stayed sealed throughout. That run establishes operational integrity only. It is not a benchmark result, and it does not rank the paths against one another.

Compute-grounded reasoning has code compute the sub-problems that have deterministic solutions before the model writes its answer. This structure offers one way to make agent decisions easier to inspect. Whether that structure improves accuracy, reliability, latency, or resource use remains unanswered here. The next phase is the scientific one. It will unseal the protected labels under a frozen analysis plan, score the completed prediction journals, and report paired comparisons with uncertainty intervals. Only that step can turn the present operational evidence into a claim about performance.

The public code is at `https://github.com/arunshar/spatial-atlas-agent`. It includes the agent, the four run modes, and their validators, but it cannot re-execute V37 by itself (Appendix A).

References

[1] Shuai An, Shesha Sai Kumar Reddy Sadu, Arun Sharma, Majid Farhadloo, and Shashi Shekhar. Discovering Super-Colocation Patterns: A Summary of Results. In *Proceedings of the 19th International Symposium on Spatial and Temporal Data*, pages 34–38, 2025. doi:10.1145/3748777.3748790.

[2] BerriAI. LiteLLM. GitHub repository, 2023. `https://github.com/BerriAI/litellm`.

[3] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, et al. SAM 3: Segment anything with concepts. In *International Conference on Learning Representations*, 2026. arXiv:2511.16719.

[4] K. Chaloner and I. Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304, 1995.

[5] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025. arXiv:2410.07095.

[6] B. Chen, Z. Xu, S. Kirmani, et al. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[7] S. Cho, R. Hachiuma, A. Badki, H. Su, B.-K. Lee, C. H. Song, S. Liu, S. Radhakrishnan, S. Kim, Y.-C. F. Wang, and M.-H. Chen. SpatialClaw: Rethinking action interface for agentic spatial reasoning. *arXiv preprint arXiv:2606.13673*, 2026. Code: https://github.com/NVlabs/SpatialClaw.

[8] S. Diao, P. Wang, Y. Lin, R. Pan, X. Liu, and T. Zhang. Active prompting with chain-of-thought for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1330–1350, 2024. arXiv:2302.12246.

[9] N. Erickson, J. Mueller, A. Shirkov, et al. AutoGluon-Tabular: Robust and accurate AutoML for structured data. *arXiv preprint arXiv:2003.06505*, 2020.

[10] Majid Farhadloo, Arun Sharma, Jayant Gupta, Alexey Leontovich, Svetomir N. Markovic, and Shashi Shekhar. Towards Spatially-Lucid AI Classification in Non-Euclidean Space: An Application for MxIF Oncology Data. In *Proceedings of the 2024 SIAM International Conference on Data Mining*, pages 616–624, 2024. doi:10.1137/1.9781611978032.71. arXiv:2402.14974.

[11] Majid Farhadloo, Arun Sharma, Shashi Shekhar, and Svetomir Markovic. Spatial Computing Opportunities in Biomedical Decision Support: The Atlas-EHR Vision. *ACM Transactions on Spatial Algorithms and Systems*, 10(3), Article 21, 36 pages, 2024. doi:10.1145/3679201.

[12] M. Feurer, K. Eggensperger, S. Falkner, M. Lindauer, and F. Hutter. Auto-Sklearn 2.0: Hands-free AutoML via meta-learning. *Journal of Machine Learning Research*, 23(261):1–61, 2022.

[13] Google. Announcing the Agent2Agent Protocol (A2A). Google Developers Blog, 2025. https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/. Protocol specification version 0.3.0 at https://a2a-protocol.org/v0.3.0/specification/.

[14] Jayant Gupta and Arun Sharma. Mining Taxonomy-aware Colocations: A Summary of Results. In *Proceedings of the 30th International Conference on Advances in Geographic Information Systems*, Article 98, 11 pages, 2022. doi:10.1145/3557915.3561034.

[15] M. Hildebrandt, H. Li, R. Koner, et al. Scene graph reasoning for visual question answering. *arXiv preprint arXiv:2007.01072*, 2020.

[16] N. Hollmann, S. Müller, and F. Hutter. Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering. In *Advances in Neural Information Processing Systems*, volume 36, pages 44753–44775, 2023. arXiv:2305.03403.

[17] D. Hudson and C. Manning. GQA: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

[19] H. Jin, F. Chollet, Q. Song, and X. Hu. AutoKeras: An AutoML library for deep learning. *Journal of Machine Learning Research*, 24(6):1–6, 2023.

[20] R. Krishna, Y. Zhu, O. Groth, et al. Visual Genome: Connecting language and vision using crowdsourced dense image annotations. *International Journal of Computer Vision*, 123:32–73, 2017.

[21] Y. Li, Y. Du, K. Zhou, et al. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[22] Y.-H. Liao, R. Mahmood, S. Fidler, and D. Acuna. Reasoning paths with reference objects elicit quantitative spatial reasoning in large vision-language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. arXiv:2409.09788.

[23] H. Lin, S. Chen, J. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang. Depth Anything 3: Recovering the visual space from any views. In *International Conference on Learning Representations*, 2026. arXiv:2511.10647.

[24] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, volume 36, pages 34892–34916, 2023.

[25] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[26] B. Settles. Active learning literature survey. *Computer Sciences Technical Report 1648*, University of Wisconsin–Madison, 2009.

[27] Arun Sharma, Jayant Gupta, and Subhankar Ghosh. Towards a Tighter Bound on Possible-Rendezvous Areas: Preliminary Results. In *Proceedings of the 30th International Conference on Advances in Geographic Information Systems*, Article 97, 11 pages, 2022. doi:10.1145/3557915.3561033.

[28] Arun Sharma, Subhankar Ghosh, and Shashi Shekhar. Physics-Based Abnormal Trajectory Gap Detection. *ACM Transactions on Intelligent Systems and Technology*, 15(5), Article 106, 31 pages, 2024. doi:10.1145/3673235.

[29] Significant Gravitas. Auto-GPT: An autonomous GPT-4 experiment. GitHub repository, 2023. https://github.com/Significant-Gravitas/AutoGPT.

[30] J. Takahashi, A. Moteki, A. Uchida, S. Masui, F. Yang, K. Uchino, Y. Song, Y. Bisk, G. Neubig, I. Kusajima, Y. Watanabe, H. Ishida, K. Nakagawa, and S. Jiang. FieldWorkArena: Agentic AI benchmark for real field work tasks. In *International Conference on Pattern Recognition (ICPR)*, 2026. arXiv:2505.19662.

[31] L. Wang, C. Ma, X. Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi:10.1007/s11704-024-40231-1.

[32] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025. arXiv:2407.16741.

[33] B. Xiao, H. Wu, W. Xu, et al. Florence-2: Advancing a unified representation for a variety of vision tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[34] D. Xu, Y. Zhu, C. Choy, and L. Fei-Fei. Scene graph generation by iterative message passing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

[35] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652, 2024. arXiv:2405.15793.

A Reproducibility and Disclosure Boundary

Code and presentation materials. The public code is available at https://github.com/arunshar/spatial-atlas-agent. The narrated demonstration is available at https://www.youtube.com/watch?v=_0O3Mz0BUiw. The presentation slides are available at https://claude.ai/artifact/4AddsBUXqpmUCNYzkemxs2. The video and slides explain the system, and the reproducible components are specified below.

Public entry points. The A2A server and task router are in `src/server.py` and `src/agent.py`. The spatial path is in `src/fieldwork/`. In that folder, `spatial.py` builds the scene graph, `perception.py` implements the strict metric bridge and the generic metric engine, `handler.py` selects the path, and `reasoner.py` implements the controller of Algorithm 1. The self-grade and the unused information-gain selector are in `src/entropy/engine.py`. The ML path is in `src/mlebench/`, and the benchmark driver that exposes the run modes is `eval_bench.py`.

Run-mode definitions. A path is selected by two options that are not freely combinable. The engine is either `scenegraph` or `metric`. The image mode is one of `correct`, `question-only`, or `shuffled`. The driver enforces four constraints. The `question-only` mode requires the `scenegraph` engine, since no image is supplied to measure. The `shuffled` mode requires the `metric` engine and an explicit control-mapping artifact. A control mapping is rejected for any image mode other than `shuffled`. The four paths in Section 8 are therefore `scenegraph+correct`, `scenegraph+question-only`, `metric+correct`, and `metric+shuffled`, as in Eq. (16). The driver turns on strict mode whenever the `metric` engine is selected.

Control-mapping contract. In the public driver, a shuffled-image run is admitted only when the mapping holds exactly 84 pairs, is a bijection, contains no fixed point, and matches the recorded digest of every source and control image (Eq. (17)). Any drift, self-pairing, repeated source, or non-bijective mapping is rejected before the run starts.

Label-free journal schema. Pre-scoring rows are appended under the versioned label-free schema `label_free_prescore_v1`. Each row stores the model's prediction text with run metadata and no ground-truth field. The writer rejects label-bearing keys anywhere in a payload. It also refuses to mix label-free and ordinary rows in one journal, and it refuses to resume a journal whose schema does not match the current run. These checks let a run complete and be inspected before any label is opened, and they check only what the run writes.

Test boundaries. The public test suite checks software behavior, and Section 8.3 reports its totals. This paper does not describe what each test checks. Passing tests establish only the behavior they exercise. They are not benchmark evidence, and they do not promote proposed behavior into a result.

What is not reproducible from this repository. The metric path needs an externally installed SpatialClaw package and a reachable GPU perception service. Every V37 arm also needs a SpatialClaw checkout and the benchmark data, because the driver loads questions and images through the SpatialClaw benchmark factory. The strict arms also need a benchmark loader for the horizontal-gap questions, and neither public SpatialClaw nor this repository includes it. The gated benchmark images and the frozen control artifact are not bundled. The combined orchestration used for V37 ran in a separate private execution environment and is not included. So V37 cannot be re-executed from the public repository alone.

Disclosure boundary. This paper omits private execution identifiers, scheduler job identifiers, filesystem locations, credential material, service registry entries, raw service logs, private control digests, and protected labels or ground truth. Their absence is a disclosure decision. It does not remove evidence that any claim in this paper depends on.

Scope of reported numbers. The counts of 323 passed and 1 deselected come from a local run of the public CI test command, and the 92.09% coverage figure covers `src/fieldwork/perception.py` only. The counts of 350 tests plus 63 subtests, and the V37 arm, row, retry, log, and finalizer-invocation counts, come from the private V37 record, which this repository does not include. The other numbers in the body are configured constants, code-level rules, and dependency versions read from the public source, plus the stated scale of a cited benchmark.

Revision note. This version removes the evaluation tables of the previous version, because no run artifact backs the values they reported. It also corrects the system description and the bibliography against the released code.